

[46] Annoy: Approximate Nearest Neighbors in C++/Python

요약

Annoy 는 Spotify 에서 개발한 대규모 ANN(Approximate Nearest Neighbor) 검색을 위한 라이브러리로, 2013 년에 발표되었다. C++로 구현되었으며 Python 바인딩을 제공하므로 다양한 환경에서 사용할 수 있다. 핵심 알고리즘은 Random Projection Trees 를 기반으로 하며, 고차원 벡터 데이터에서 빠른 근사 최근접 이웃 검색을 가능하게 한다.

Annoy 의 주요 특징은 메모리 효율성과 빠른 쿼리 속도이다. Random projection tree 구조를 이용하여 고차원 공간을 재귀적으로 분할하고, 이진 파일 형태로 인덱스를 저장할 수 있다. 메모리 매핑을 지원하므로 매우 큰 데이터셋도 효율적으로 처리할 수 있으며, 실시간 검색 애플리케이션에서 광범위하게 사용되었다.

Annoy 의 Random Projection Trees 는 고차원 벡터를 임의로 선택된 두 점을 지나는 하이퍼플레인으로 분할하는 방식으로 작동한다. 이 접근방식은 구현이 간단하면서도 효과적이며, 실무 환경에서 높은 성능을 보여준다. 검색 과정에서는 리프 노드 근처의 후보들을 탐색하고, 필요시 추가 트리를 검사하여 정확도를 조정할 수 있다.

메모리 효율성 측면에서 Annoy 는 각 벡터를 부동소수점으로 저장하며, 인덱스 구조도 매우 컴팩트하다. 이는 모바일 기기나 엷지 디바이스에서도 대규모 벡터 데이터를 처리할 수 있게 한다. 특히 음악 추천 시스템에서 백만 개 이상의 곡을 실시간으로 검색해야 하는 Spotify 의 요구사항을 충족하기 위해 설계되었다.

Exqutor 와의 관계를 살펴보면, Annoy 는 유사성 검색 인덱싱 기술의 발전을 보여주는 중요한 작업이다. Exqutor 는 유사성 쿼리의 카디널리티 추정을 다루는데, 이는 ANN 검색 결과에 기반한 시스템에서 쿼리 최적화를 위해 중요하다. 따라서 Annoy 와 같은 효율적인 ANN 구조를 이해하는 것은 Exqutor 의 카디널리티 추정 모델이 실제 검색 시스템에서 어떻게 적용되는지 이해하는 데 필수적이다.

상세분석

46.1 Random Projection Trees의 구조

Random Projection Trees (RP-Trees)는 Annoy의 핵심 자료구조이다. 이 구조는 고차원 벡터 공간을 재귀적으로 분할하는 방식으로 작동한다. 구체적으로, 각 내부 노드에서는 임의로 선택된 두 점을 지나는 하이퍼플레인을 사용하여 벡터들을 두 그룹으로 나눈다.

하이퍼플레인의 선택 방식은 다음과 같다. 현재 노드에 할당된 벡터 집합에서 임의로 두 점 p_1 과 p_2 를 선택하고, 이 두 점의 중점을 지나면서 $p_2 - p_1$ 벡터에 수직인 하이퍼플레인을 정의한다. 이 하이퍼플레인을 기준으로 각 벡터를 분류하여 왼쪽 또는 오른쪽 자식 노드로 할당한다.

이러한 방식의 장점은 계산의 간단성과 효율성에 있다. 하이퍼플레인 정의에 필요한 계산량이 적으므로, 대규모 데이터셋에 대해서도 빠르게 인덱스를 구축할 수 있다. 또한 임의성으로 인해 자동적으로 어느 정도 로드 밸런싱이 이루어지므로, 트리가 한쪽으로 치우치지 않도록 한다.

리프 노드에 도달하면 해당 노드의 모든 벡터들이 저장된다. 리프 노드의 크기는 하이퍼파라미터로 조정할 수 있으며, 이는 메모리 사용량과 쿼리 속도 간의 트레이드오프를 제어한다.

46.2 검색 알고리즘과 근사 특성

Annoy의 검색 프로세스는 쿼리 벡터 q 가 주어졌을 때, RP-Tree를 따라 루트에서 리프까지 내려가는 과정으로 시작된다. 각 내부 노드에서 q 와 하이퍼플레인의 위치 관계를 판단하여 적절한 자식 노드를 선택한다.

리프 노드에 도달한 후, 해당 리프 내의 모든 벡터들과 q 사이의 거리를 계산하고, 상위 k 개를 후보로 선택한다. 기본적인 검색은 여기서 종료되지만, 더 높은 정확도를 원할 경우 추가 탐색이 가능하다.

추가 탐색 과정에서는 현재 리프 노드의 형제 노드나 근처의 다른 리프 노드들도 검사한다. Annoy에서는 `search_k`라는 파라미터를 통해 검사할 후보의 개수를 제어할 수 있다. `search_k` 값이 크면 정확도가 높아지지만 쿼리 속도는 느려진다.

근사 특성은 확률론적 보장이 아니라 경험적 성능에 기반한다. 실제로 많은 응용 분야에서 95% 이상의 정확도를 달성하면서도 정확한 브루트포스 탐색보다 10 배 이상 빠른 성능을 제공한다.

46.3 메모리 효율성과 파일 저장

Annoy 의 또 다른 핵심 특징은 메모리 효율성이다. 인덱스를 바이너리 파일로 저장할 때, 각 벡터는 실수 배열로 저장되며, 트리 구조는 매우 간결하게 인코딩된다. 일반적으로 각 차원당 4 바이트(float32) 또는 8 바이트(float64)만 사용한다.

메모리 매핑(Memory Mapping) 기능을 지원하므로, 디스크의 인덱스 파일을 직접 메모리에 매핑하여 사용할 수 있다. 이는 매우 큰 데이터셋의 경우 메인 메모리의 크기에 제약받지 않고도 검색을 수행할 수 있다는 의미이다.

인덱스 파일의 구조는 헤더 정보(벡터 차원수, 트리 개수 등)와 인덱스 데이터로 구성된다. 트리 구조는 배열 기반의 이진 트리 표현을 사용하여, 포인터 오버헤드를 최소화한다.

46.4 하이퍼파라미터 조정

Annoy의 성능은 여러 하이퍼파라미터에 의해 영향을 받는다. 첫째, `n_trees`는 구축할 RP-Tree의 개수이다. 더 많은 트리를 사용하면 정확도가 높아지지만 메모리 사용량과 인덱싱 시간이 증가한다.

둘째, `search_k`는 검색 시 탐색할 후보 벡터의 최대 개수를 제어한다. `search_k` 값이 크면 정확도가 높아지지만 쿼리 응답 시간이 길어진다. 보통 `search_k > n_trees * 트리의 리프 노드 개수` 범위에서 조정된다.

셋째, `leaf_size`는 리프 노드에 저장할 최대 벡터 개수를 제어한다. 작은 값은 더 정확한 검색을 제공하지만 더 많은 메모리를 사용한다.

46.5 실무 적용 및 성능

Spotify는 음악 추천 시스템에서 Annoy를 광범위하게 사용했다. 수백만 개의 곡을 벡터로 표현하고, 사용자가 현재 듣고 있는 곡과 유사한 곡들을 실시간으로 검색해야 한다. Annoy는 이러한 요구사항을 효과적으로 충족한다.

성능 측면에서, Annoy는 일반적으로 다음과 같은 특성을 보여준다. 인덱싱 속도는 초당 백만 개 벡터 정도의 속도로 빠르며, 쿼리 응답 시간은 밀리초 단위의 초저지연(ultra-low latency)을 제공한다. 메모리 사용량은 벡터 데이터 자체의 크기에 가깝다.

추가 제기 문제

1. **정확도 보장의 부재:** Annoy 는 근사 알고리즘이므로 항상 실제 최근접 이웃을 찾는다는 보장이 없다. 애플리케이션에서 이러한 근사성이 허용 가능한 수준인지 검증이 필요하다.
2. **벡터 추가의 어려움:** Annoy 는 정적 인덱스이므로, 새로운 벡터를 추가하려면 전체 인덱스를 재구축해야 한다. 동적 데이터셋에는 부적합할 수 있다.
3. **고차원에서의 성능 저하:** Random projection 은 고차원에서 curse of dimensionality 를 완전히 극복하지는 못한다. 벡터의 차원이 매우 높으면 성능이 저하될 수 있다.
4. **카디널리티 추정과의 통합:** Annoy 같은 ANN 인덱스를 사용하는 시스템에서, 유사성 검색의 결과 크기를 정확하게 예측하려면 별도의 카디널리티 추정 기법이 필요하다. 이것이 Exqutor 가 다루는 핵심 문제이다.